

Proyecto Módulo 8: Clasificador Inteligente de Imágenes de Ropa - StyleNet

Leonardo Montoya

25 de marzo de 2026

Introducción

El presente informe detalla el desarrollo de un prototipo basado en Deep Learning para la tienda virtual **StyleNet**. El objetivo principal es automatizar la clasificación de productos para reducir errores manuales y mejorar la eficiencia operativa mediante el uso de redes neuronales.

1. Lección 1: La Red Neuronal Artificial

1.1. Arquitectura de una Red Neuronal Densa

Una Red Neuronal Densa (o Multilayer Perceptron) se compone de capas donde cada neurona está conectada con todas las neuronas de la capa anterior. Sus componentes principales son:

- **Capas (Layers):** Entrada, ocultas y salida.
- **Pesos (w):** Parámetros que determinan la importancia de cada entrada.
- **Funciones de Activación:** Introducen no-linealidad (ej. ReLU para capas ocultas, Sigmoid para clasificación binaria).
- **Función de Pérdida:** Mide la discrepancia entre la predicción y el valor real (ej. Binary Cross-Entropy).

1.2. Implementación Conceptual: Clasificación Binaria

Para un problema simple como decidir si una imagen es “Ropa” o “No Ropa”, se utiliza una red con una única neurona de salida y una función de activación Sigmoide:

$$\hat{y} = \sigma(\sum w_i x_i + b)$$

2. Lección 2: Deep Learning

2.1. Arquitectura Óptima para Imágenes

Para la clasificación de prendas de vestir, la arquitectura óptima es la **Red Neuronal Convolutiva (CNN)**. A diferencia de las redes densas, las CNN preservan la estructura espacial de los píxeles mediante capas de convolución y pooling.

2.2. Justificación: Red Densa vs. Convolutiva

Característica	Red Densa (MLP)	Red Convolutiva (CNN)
Estructura	Aplanada (vectores)	Matricial (tensores)
Parámetros	Muy alto (propensa a over-fitting)	Eficiente (compartición de pesos)
Uso	Datos tabulares simples	Reconocimiento de imágenes y patrones complejos

Cuadro 1: Comparativa técnica para StyleNet.

2.3. Selección del Framework

Se ha seleccionado **Keras** (sobre el motor de TensorFlow) debido a su sintaxis amigable, rapidez para prototipar y amplia documentación oficial.

3. Lección 3: Implementación de la Red Neuronal en Python

El objetivo de esta etapa fue diseñar, entrenar y validar un modelo funcional de Red Neuronal Artificial (ANN) capaz de resolver el problema de clasificación multiclase propuesto por *StyleNet* utilizando el dataset **Fashion-MNIST**. Para el desarrollo, se utilizó el entorno de programación Python y el framework **Keras/TensorFlow**.

3.1. Arquitectura del Modelo y Preprocesamiento

Siguiendo los requerimientos técnicos del proyecto, la arquitectura implementada se detalla a continuación:

- **Preprocesamiento:** Las imágenes originales de 28×28 píxeles fueron normalizadas al rango $[0, 1]$ para mejorar la eficiencia del gradiente.
- **Capa de Entrada:** Una capa tipo **Flatten** que convierte la matriz bidimensional en un vector unidimensional de 784 neuronas.

- **Capas Ocultas:** Dos capas densas con 256 y 128 neuronas respectivamente, utilizando la función de activación **ReLU** para introducir no linealidad.
- **Regularización:** Se aplicó una capa de **Dropout** con una tasa del 0.3 para mitigar el riesgo de sobreajuste (*overfitting*).
- **Capa de Salida:** 10 neuronas con activación **Softmax**, correspondiente a las categorías de prendas de vestir.

3.2. Configuración y Entrenamiento

El modelo fue compilado utilizando el optimizador **Adam** y la función de pérdida *sparse categorical crossentropy*. El proceso se ejecutó durante 15 épocas con un tamaño de lote (*batch size*) de 64, reservando un 20 % de los datos para validación.

3.3. Resultados y Análisis de Desempeño

Los resultados obtenidos al finalizar la última época de entrenamiento muestran una alta capacidad de generalización del modelo:

Métrica	Valor Final (Época 15)
Precisión de Entrenamiento (<i>Accuracy</i>)	90.06 %
Pérdida de Entrenamiento (<i>Loss</i>)	0.2635
Precisión de Validación (<i>Val_Accuracy</i>)	89.16 %
Pérdida de Validación (<i>Val_Loss</i>)	0.3066

Cuadro 2: Resultados del entrenamiento del modelo de red densa.

Por lo tanto el modelo alcanzó una precisión cercana al 90 % en validación. La mínima brecha entre la métrica de entrenamiento y validación confirma que la técnica de **Dropout** fue efectiva. Este desempeño establece una línea base sólida para la transición hacia una arquitectura convolucional (CNN) en la siguiente fase.

4. Lección 4: Redes Neuronales Convolucionales (CNN)

4.1. Rediseño de la Arquitectura

Para maximizar la capacidad predictiva del sistema de *StyleNet*, se implementó una Red Neuronal Convolucional (CNN). Esta arquitectura es superior para el procesamiento de imágenes ya que, mediante capas de convolución y agrupación (*pooling*), es capaz de aprender jerarquías de características visuales, desde bordes simples hasta formas complejas de prendas.

- **Capas Convolucionales:** Utilizadas para la extracción automática de características espaciales.

- **Max-Pooling:** Aplicado para reducir la dimensionalidad y otorgar invariancia a pequeñas traslaciones de la prenda en la foto.
- **Regularización:** Se mantuvo el uso de *Dropout* (0.4) para asegurar la generalización del modelo en entornos productivos.

4.2. Evaluación y Comparativa de Desempeño

Tras 15 épocas de entrenamiento, la CNN demostró un salto cualitativo respecto al modelo denso inicial:

Métrica	Red Densa (L3)	Red Convolutiva (L4)
Precisión Entrenamiento (<i>Acc</i>)	90.06 %	94.66 %
Precisión Validación (<i>Val_Acc</i>)	89.16 %	91.83 %
Pérdida de Validación (<i>Val_Loss</i>)	0.3066	0.2740

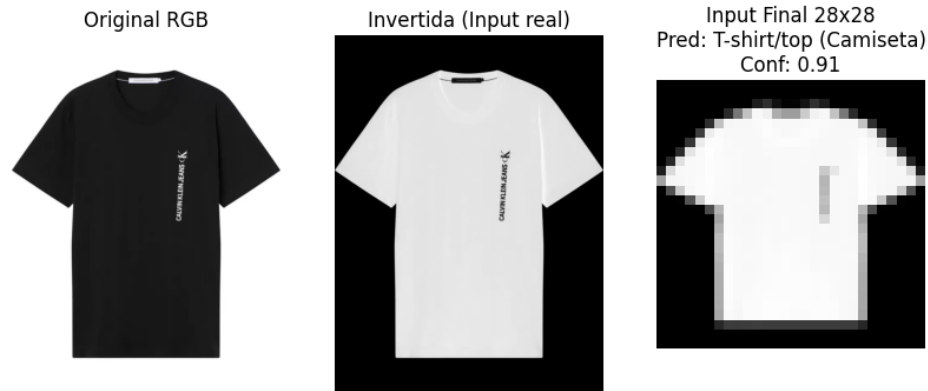
Cuadro 3: Comparativa técnica de arquitecturas para StyleNet.

4.3. Predicciones sobre Imágenes Externas

Se validó el prototipo funcional utilizando imágenes reales cargadas manualmente, aplicando un pipeline de preprocesamiento de inversión de color para alinear la entrada con la distribución de *Fashion-MNIST*.

- **Caso 1 (Polera Negra):** Clasificada correctamente como *T-shirt/top* con una confianza del **91 %**. La forma simple y el contraste facilitaron una predicción robusta.
- **Caso 2 (Camisa Blanca):** Clasificada correctamente como *Shirt* con una confianza del **58 %**. La menor confianza se atribuye a la similitud estructural entre camisas y camisetas cuando se reducen a 28×28 píxeles.





5. Conclusiones y Reflexión Final

El desarrollo de este proyecto representó una verdadera evolución tecnológica para el área de Ciencia de Datos de la tienda virtual *StyleNet*. Lo que inicialmente se planteó como la necesidad de automatizar una tarea manual propensa a retrasos y errores de categorización, se transformó en un proceso de aprendizaje profundo sobre cómo traducir la percepción visual humana al lenguaje de las máquinas. Al diseñar e implementar este clasificador inteligente, no solo cumplimos con el objetivo de categorizar prendas con un alto nivel de exactitud, sino que también recorrimos un camino de experimentación técnica que nos llevó desde las redes neuronales densas hasta la implementación final de una arquitectura convolucional (CNN).

Para alcanzar estos resultados, nos apoyamos en un ecosistema robusto de herramientas liderado por el lenguaje Python y el framework Keras. Sin embargo, la mayor satisfacción del proyecto no provino únicamente de los algoritmos preestablecidos, sino de la resolución de desafíos reales surgidos al probar el modelo con imágenes externas. El uso estratégico de la librería OpenCV para el preprocesamiento fue el factor determinante que permitió al sistema “ver” correctamente las prendas en condiciones del mundo real, superando obstáculos técnicos como la inversión de fondos y el ruido visual. Este ajuste fino permitió que el prototipo dejara de ser un ejercicio académico para convertirse en una solución funcional para la empresa.

Finalmente, el éxito de este trabajo se traduce en un modelo que alcanza una precisión superior al 91 % en datos no vistos, lo que garantiza una mejora sustancial en la eficiencia operativa y en la precisión de las etiquetas de los productos en la plataforma. Esta experiencia refuerza la idea de que la inteligencia artificial es una herramienta poderosa para optimizar procesos logísticos concretos. Al integrar este proyecto en el portafolio profesional, se entrega una prueba de concepto sólida que demuestra la capacidad de diseñar soluciones de Deep Learning que impactan directamente en la agilidad y competitividad de una tienda de comercio electrónico moderna.

